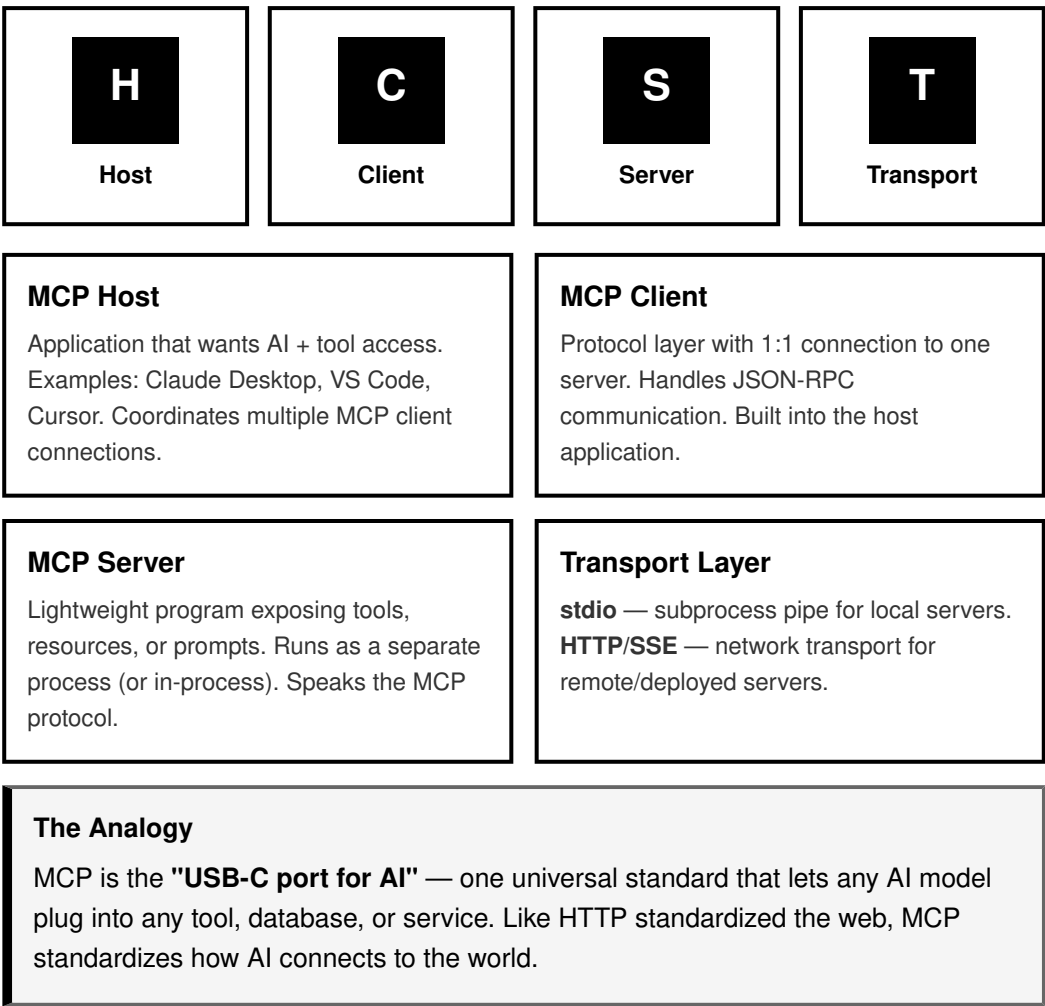


BUILDING AI AGENTS WITH MCP

The HTTP Moment of AI — O'Reilly Live Training Cheatsheet

Model Context Protocol · Claude Agent SDK · FastMCP · Deployment | 2025

1. WHAT IS MCP?



2. THE 4 MCP CAPABILITIES

Tools — Model-controlled

Functions the AI decides to call. Like function calling — LLM triggers execution.

```
@mcp.tool()
```

Resources — Application-controlled

Data the host app exposes to context. Files, DB rows, API data. URI-addressed.

```
@mcp.resource("uri://")
```

Prompts — User-controlled

Reusable prompt templates users invoke. Slash commands in Claude Desktop.

```
@server.list_prompts()
```

Sampling — Server-initiated

Server requests LLM completions. Enables agentic behavior inside servers.

```
session.create_message()
```

3. MCP LIFECYCLE

1

Initialize

Client sends capabilities + version. Server responds with its capabilities.

2

Handshake

Client sends notifications/
initialized .
Both sides ready.

3

Operate

Tool calls, resource reads, prompt requests, sampling — all via JSON-RPC 2.0.

4

Shutdown

Client sends notifications/
closed . Server cleans up.

4. CORE CODE PATTERNS

FastMCP Server Pattern

```
#!/usr/bin/env -S uv run --script
# /// script
# requires-python = ">=3.9"
# dependencies =
#   ["mcp[cli]>=1.0.0"]
# ///

from mcp.server.fastmcp import
FastMCP

mcp = FastMCP("server-name")

@mcp.tool()
def my_tool(param: str) -> str:
    """Description visible to
    LLM"""
    return result

@mcp.resource("data://{key}")
def get_resource(key: str) ->
str:
    """Resource description"""
    return load_data(key)

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Claude Agent SDK Pattern

```
import asyncio
from claude_agent_sdk import (
    Agent, tool,
    create_sdk_mcp_server
)

# Custom tool with decorator
@tool
def my_tool(input: str) -> str:
    """Tool available to the
    agent"""
    return process(input)

# In-process MCP server (no
subprocess)
server =
create_sdk_mcp_server("analysis")

@server.tool()
def analyze(data: str) -> str:
    return run_analysis(data)

# Run the agent
agent = Agent(
    name="helper",
    instructions="System
instructions",
    tools=[my_tool],
    model="claude-sonnet-4-5"
)
asyncio.run(agent.run("Task"))
```

JSON-RPC 2.0 — Tool Call

```
{
  "jsonrpc": "2.0",
  "id": "req-1",
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {"city":
"Lisbon"}
  }
}

// Response
{
  "jsonrpc": "2.0",
  "id": "req-1",
  "result": {
    "content": [
      {"type": "text", "text":
"22°C, Sunny"}
    ]
  }
}
```

Security: PreToolUse Hook

```
from claude_agent_sdk import
PreToolUse

def block_dangerous(tool_use):
    """Block rm -rf and similar
    commands"""
    if tool_use.tool_name ==
"Bash":
        cmd =
tool_use.input.get("command", "")
        dangerous = ["rm -rf",
"sudo rm",
                    "format", "> /
dev/"]
        for d in dangerous:
            if d in cmd:
                return {"type":
"block",
                    "reason":
"Blocked"}
        return {"type": "allow"}

# Attach to agent
agent = Agent(

hooks=[PreToolUse(block_dangerous)]
,
    ...
)
```

5. COURSE DEMO PROGRESSION

8 Hands-On Demos

#	Demo	What You Build	Key Concept
00	Intro to Agents	Agent loop fundamentals	When agents vs simple LLM calls
01	Intro to MCP	First FastMCP server + client	stdio transport, MCP Inspector
02	Chat App (Claude API)	Full chat app with tool use	Tools + Resources + Prompts
03	Filesystem Agent	File reader with SDK + MCP	In-process server, hooks, permissions
04	Tabular Data Query	CSV querying via MCP tools	pandas ops as MCP tools
05	Link Checker Agent	Markdown link health auditor	Automation: 4-tool MCP server
06	Deploy to Vercel	Data analysis app + chat UI	FastAPI + SSE streaming + Vercel
07	Hacks & Tips	MCP builder skill workflow	AI-assisted MCP server creation

6. SECURITY CHECKLIST & COMMON GOTCHAS

Security Checklist

- Validate file paths — prevent directory traversal (`..` , leading `/`)
- Use `PreToolUse` hooks to block dangerous Bash commands
- Principle of least privilege — grant only needed tools
- Use HTTPS for remote server transport
- Audit log all sensitive operations
- Rate-limit tool calls in production
- Never commit `.env` files (API keys)

Common Gotchas

- All MCP ops are **async** — don't forget `await`
- Use **absolute paths** — relative paths break when server runs from a different directory
- Resource URIs must be valid: `file:///path` not `/path`
- "Method not found" → server doesn't implement that capability
- "Invalid params" → check your JSON schema / type hints
- Vercel hobby plan: 10s timeout — upgrade to Pro for Agent SDK cold starts

7. KEY COMMANDS & TOOLS

Running Demos with UV

```
# UV handles all dependencies automatically
uv run demos/01-introduction-to-mcp/mcp_server.py

# Test any MCP server interactively
mcp dev demos/01-introduction-to-mcp/mcp_server.py
# Opens browser: tool test + message inspector

# Enable debug logging in any server
import logging
logging.basicConfig(level=logging.DEBUG)

# Check Claude Desktop logs
~/config/Claude/logs/
```

Claude Desktop Config

```
// ~/.config/Claude/claude_desktop_config.json
{
  "mcpServers": {
    "my-server": {
      "command": "uv",
      "args": [
        "run",
        "/absolute/path/to/server.py"
      ]
    }
  }
}

// After editing: restart Claude Desktop
// Use absolute paths – relative paths fail
```

8. ECOSYSTEM HACKS & RESOURCES

Build MCP Servers with AI

Use Claude Code to scaffold servers. Paste MCP SDK docs + requirements into the prompt. Iterate on one capability at a time.

Tip: Use Context7 MCP server for always-up-to-date SDK docs inside Claude Code.

LLM-Ready Repo Data

gitingest.com — Replace `github.com` with `gitingest.com` in any repo URL to get a single LLM-ready text dump of the whole repo.

deepwiki.com — Instant docs page for any GitHub repo.

Doc Indexing

llmstxt.org — Standard for AI-readable doc index files. Add `/llms.txt` to any site to make it LLM-navigable.

Check if a site has it:
`site.com/llms.txt`

Key References

Resource	URL
MCP Specification	modelcontextprotocol.io/specification
MCP Python SDK	github.com/modelcontextprotocol/python-sdk
Claude Agent SDK (Python)	github.com/anthropics/claude-agent-sdk-python
Community MCP Servers	github.com/modelcontextprotocol/servers
MCP Security Best Practices	modelcontextprotocol.io/specification/.../security_best_practices
Course Repository	github.com/automata-learning-lab/mcp-course

KEY TAKEAWAYS

- **MCP = USB-C for AI:** One protocol connects any AI model to any tool, database, or service
- **4 Capabilities:** Tools (model runs), Resources (app exposes), Prompts (user invokes), Sampling (server asks LLM)
- **FastMCP:** Use `@mcp.tool()` decorators — Python type hints auto-generate the JSON schema
- **In-process servers:** `create_sdk_mcp_server()` runs MCP tools in the same Python process — no subprocess overhead
- **Security first:** Always validate paths, use `PreToolUse` hooks, grant least privilege, log everything
- **Deploy anywhere:** FastAPI + Claude Agent SDK + SSE streaming = production-ready agent in <200 lines
- **UV handles deps:** Inline `# /// script` headers mean zero setup — `uv run server.py` just works